Mini-Project 3: Collision Detection

Goals

In this mini-project you will build on the code you developed for mini-project 2 to implement a collision detection algorithm. Concepts discussed from lecture which will be applied in this mini-project include:

- Plotting and annotation
- Array functions and indexing
- Custom functions

Motivation: Collision Detection

How does your car know to deploy the airbags when you get into an accident? The answer is likely that it monitors readings from an accelerometer and deploys the airbags if it detects very rapid changes in acceleration.

In this mini-project, you will develop an application to monitor for rapid acceleration or deceleration and generate a report indicating when possible collision events (or unsafe levels of acceleration) occurred.

In this mini-project, we will analyze a sensor-log after-the-fact. In future mini-projects, we'll extend this to a *real-time* monitoring system.

Steps

Step 1: Developing your analysis pipeline

Step 1 will guide you through developing your own data analysis and report-generation pipeline.

Step 1.1: Data Collection

Create a copy of the data collection script you wrote for mini-project 1 and use it to collect data from a trial where your phone undergoes slow, then rapid, acceleration. Here are a couple example activities:

- While holding your phone, slowly crouch down and prepare to jump. Wait a few seconds and then jump as high as possible.
- Walk forward slowly and then break into a sprint as fast as possible for a couple seconds. Slowly decelerate to a fast jog and then come to as abrupt a stop as possible.

Try to keep your phone as steady as possible to avoid confounding effects from small movements in your phone.
- Slowly wave your phone back-and-forth in front of you a few times. Then wave it in the opposite direction as rapidly as possible.

To make sure you can detect the events of interest, set the sampling rate to around 50Hz.

Save your results to a file “sensorData.mat.”

Step 1.2: Detecting Rapid Acceleration Events

Your goal in this step is to develop an algorithm which can identify the timestamps of samples in your sensor log where acceleration was unusually rapid (e.g. potentially dangerous). You must create a function with the signature:

```
[ts, A, avgA] = detectRapidAcceleration( sensorData, threshold )
```

The input argument “sensorData” is an Nx4 array where N is the number of samples, columns 1-3 are measured X, Y, Z, values, the 4-th column is the timestamp, and threshold is a parameter used to detect rapid acceleration events.

The output of your function should be two arrays:

- ts - an array containing the timestamps of sensor readings for which the *resultant acceleration* exceeds or is strictly greater than the “threshold.” (For *resultant acceleration*, use the same formula from Miniproject 2).
- A - an array containing the corresponding *resultant acceleration* values
- avgA - a scalar containing the average resultant acceleration during rapid acceleration events (as determined by “threshold”). avgA should be 0.0 if no rapid acceleration events have been detected.

Here is an example of how the outputs might look like:

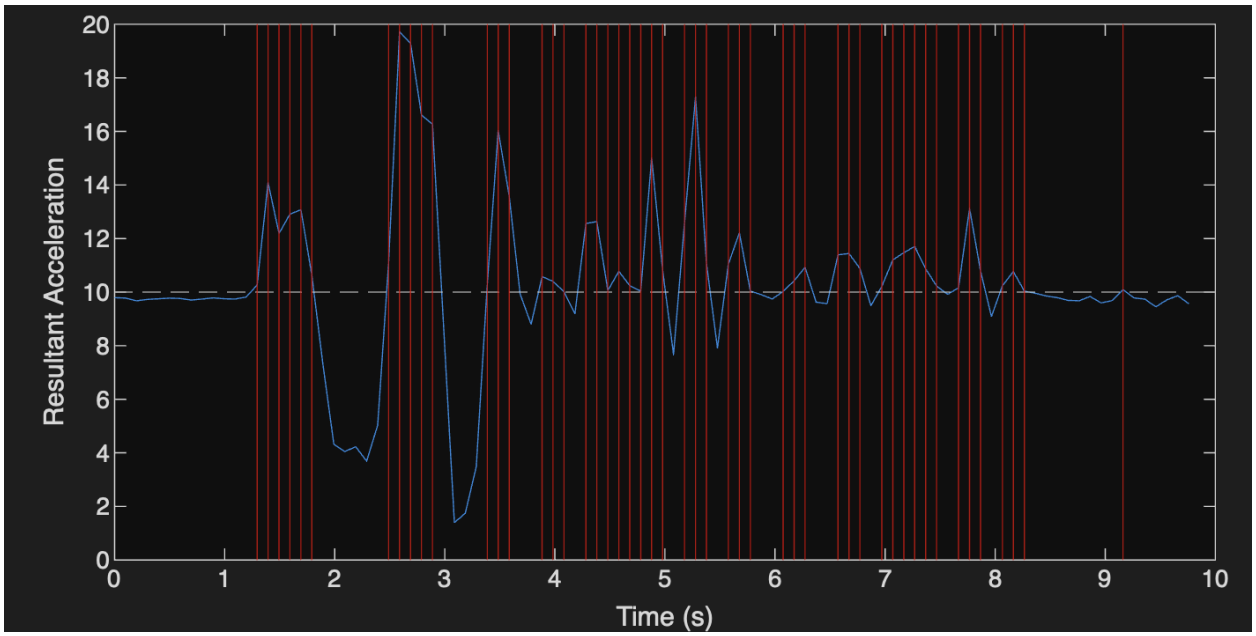
ts	: [0.4, 0.5, 0.8]	% These values are in seconds
A	: [3.45, 4.5, 7.0]	% These are our resultant accelerations
avgA	: 4.98333	% This is the average resultant acceleration

Step 1.3: Visualizing Rapid Acceleration Events

Create a new script “analyzeData.m” which loads the saved “sensorData.mat” file created in step 1.1 above and create a plot showing the resultant acceleration over time for the entirety of your recording. Using the function you wrote in step 1.2, add vertical red lines (using **xline** - https://www.mathworks.com/help/matlab/ref/xline.html) on the plot at samples corresponding to

rapid acceleration events. Plot a dashed line at the y-axis value corresponding to the threshold you used (using **yline** - https://www.mathworks.com/help/matlab/ref/yline.html).

The final plot should look something like this with Threshold = 10.0



Repeat your same data collection experiment three different times (or perform three different activities) and produce three corresponding plots. Include each plot in your lab report along with a description of the activity performed. Were you able to use the same threshold for each plot?

Step 2: Peer Validation

As in mini-project 2, trade data from one of your collection runs to a partner. Using the data received from your partner, reproduce their plot showing rapid acceleration events. Include the plot and your partner name in your lab report.

What to Submit

Submit your “detectRapidAcceleration” function and “analyzeData.m” script to the autograders on Canvas.

Submit your lab report including:

- A brief summary of your work

- Three plots of your acceleration event measurements with thresholds and rapid-acceleration events clearly marked.
- An analogous plot for your partner's data along with the name of your partner.

Rubric

- **Code**:
  - Function detectRapidAcceleration runs without errors.
  - Correctly generates the timestamps, computes the accelerations and the average acceleration.
  - Script analyzeData runs without errors and produces the requested plots.
- **Plots**:
  - Plots are clear
  - Plots contain all the vertical lines at the points of rapid acceleration
  - Plots contain a horizontal line at the given threshold
  - Both plots for you and your partner's data are generated using your own code
  - Must include partner's name in the report
- **Reflections**
  - Reflection meaningfully discusses AI results and how results were verified, and/or commentary on if it greatly differs from functions being used in class, if AI was used